KriSemi

Convolution Encoder

MATLAB -Simulink

sarathkc@krisemidesigntech.com
10-21-2026

MATLAB and Simulink Exploration of a Convolutional Encoder

1. Objective

The objective of this activity was to understand the operation of a convolutional encoder using MATLAB and Simulink, apply different input bit sequences, and verify that the encoded outputs match the expected results.

The activity establishes a reference implementation that can subsequently be used to develop and verify an equivalent RTL implementation and C program.

The work was divided into the following tasks:

1. Configure and understand the Convolutional Encoder block in Simulink.
2. Generate input bit sequences and observe the encoded outputs.
3. Develop a MATLAB reference model using the **convenc()** function.
4. Verify the Simulink encoder against the MATLAB reference model.
5. Perform automated testing using random input sequences and predefined test patterns.
6. Prepare for RTL and C implementation of the verified encoder functionality.

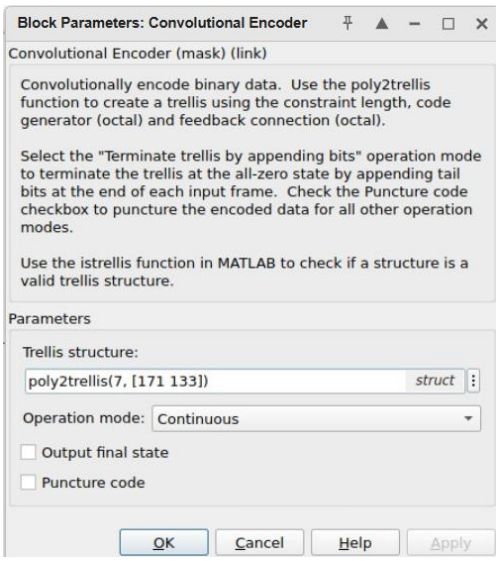
2. Convolution Encoder Overview

A convolutional encoder is a channel-coding block that introduces redundancy into an input bitstream. This redundancy can help a receiver detect and correct transmission errors when used with an appropriate decoding algorithm.

Unlike a simple combinational encoder, a convolutional encoder has memory. Its output depends on the current input bit and the encoder's previous state.

For this activity, a rate of ½ is configured in Simulink

2.1 Convolutional Encoder parameter



Parameter	Configuration
Constraint length	7
Generator polynomials	171 and 133 (octal)
Coding rate	1/2
Number of input bits per symbol	1
Number of output bits per symbol	2
Operation mode	Continuous
Puncturing	Disabled
Output final state	Unchecked

The trellis structure was defined in MATLAB using :
trellis = poly2trellis(7, [171 133]);

The constraint length of 7 corresponds to a memory order of 6 for this encoder. The encoder's state represents its memory, and the generator polynomials determine how the current input and stored state contribute to the output bits.

For a rate-1/2 encoder, each input bit generates two output bits.

Therefore, for an input sequence of 10 bits, the expected encoded output length is:

$$N_{out} = N_{in} / R$$

where:

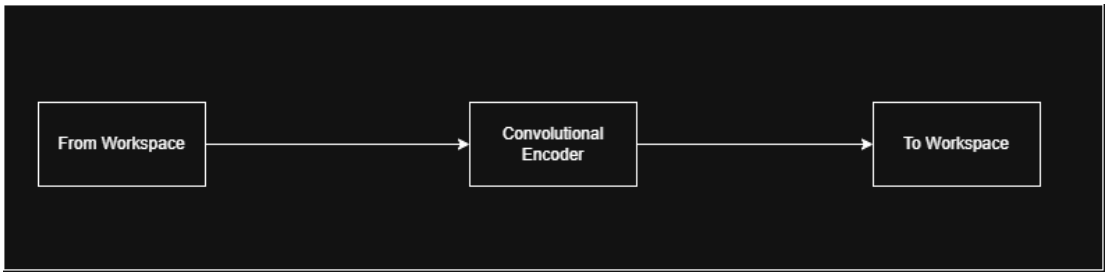
- Nout is the output sequence length.
- Nin is the input sequence length.
- R is the coding rate.

➔ N_{in} = 10, Rate(R) = ½
➔ N_{out} = 10/(½) = 20

This calculation assumes that no additional termination or tail bits are appended.

3. Simulink Model Development

A Simulink model was created using three blocks connected in sequence.



3.1 From Workspace

The From Workspace block supplied the input bit sequence to the encoder. The input was represented as a MATLAB timeseries object, with one bit supplied at each sample time.

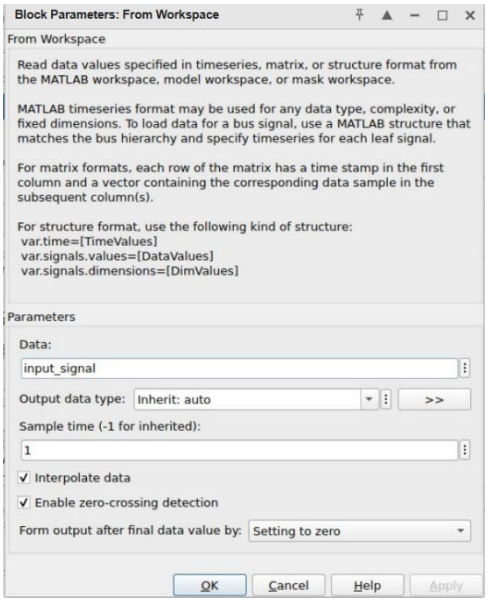
Example:
MATLAB Terminal Commands

```
input_bits = [1; 0; 1; 1; 0; 0; 1; 0; 1; 1];

input_signal = timeseries(input_bits, (0:9)');
```

The variable input_signal was configured as the data source for the From Workspace block.

Configuration:



3.2 Convolution Encoder

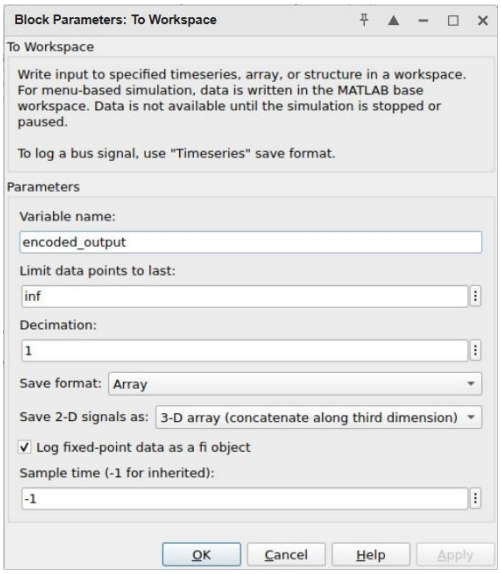
The Convolutional Encoder block was configured with the trellis corresponding to constraint length 7 and generator polynomials 171 and 133 (octal).

The block processed the input sequence and generated two encoded output bits per input bit.

3.3 To Workspace

The To Workspace block stored the encoded output in the MATLAB workspace.The output variable was configured as: encoder_output

The output was saved in Array format and accessed through the simulation output object: out.encoded_output

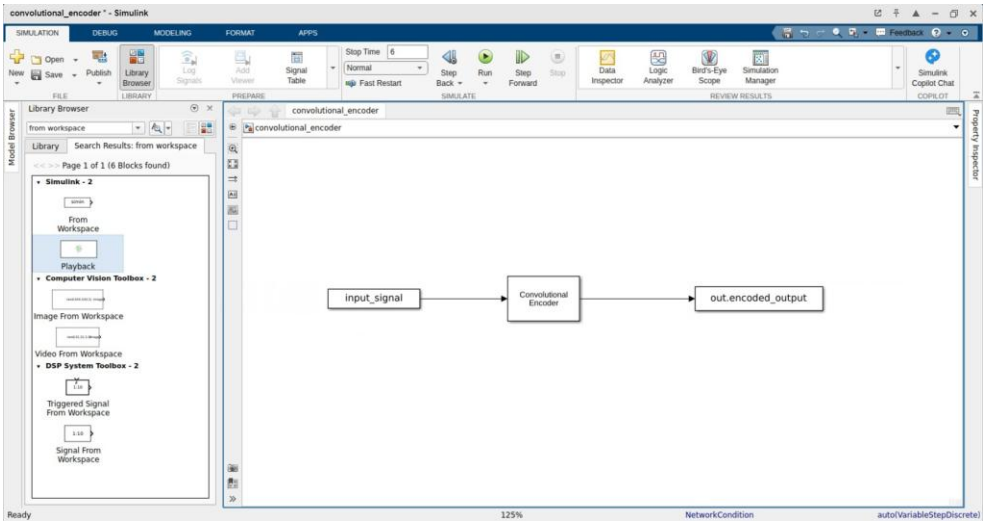


For a 10-bit input sequence, the output array contained 10 rows and 2 columns. Each row represented the pair of encoded output bits associated with an input sample.

The output was converted into a single serial bitstream for comparison with the MATLAB reference output.

```
simulink_bits = reshape(out.encoded_output.', [], 1);
```

Simulink Model:



4. MATLAB Reference Model

To independently verify the Simulink implementation, MATLAB's `convenc()` function was used as the reference encoder.

The reference trellis was created using the same encoder parameters:

```
< / > matlab

trellis = poly2trellis(7, [171 133]);
```

The input sequence was encoded using:

```
</> matlab

reference_bits = convenc(input_bits, trellis);
```

A result of 1 indicated that both output sequences matched bit-for-bit. A result of 0 indicated a mismatch.

4.1 Initial Verification

During the initial comparison, the Simulink model and MATLAB reference model used different input sequences. The comparison failed because the inputs were not identical.

The issue was identified by inspecting both input sequences.

```
input_bits = test_inputs(:,1);
nput_signal = timeseries(input_bits, (0:9)');
```

The Simulink input was then updated to use the same sequence as the reference model:After rerunning the simulation, the Simulink output matched the MATLAB reference output.

Result PASS — the output sequences matched for the corrected test case.

This highlighted the importance of applying identical input data when comparing two implementations

```
>> input_bits = [1; 0; 1; 1; 0; 0; 1; 0; 1; 1];
>> input_signal = timeseries(input_bits, (0:9)');
>> out.encoded_output

ans = 11×2
     1     1
     1     0
     0     0
     1     0
     0     1
     0     1
     1     1
     1     1
     1     0
     0     1
     0     1
```

Fig.1. Result of the first test

Extended Learning:

5. Automated Random Testing

After the initial test case passed, automated testing was performed using 50 randomly generated input sequences

5.1 Input Generation

A reproducible set of random binary sequences was generated using:

```
rng(42);
num_tests = 50;
input_length = 10;
test_inputs = randi([0 1], input_length, num_tests);
```

Each column of test_inputs represented a separate 10-bit input sequence.

The random seed ensured that the same set of test sequences could be regenerated

5.2 Reference Output Generation

The expected output for each test sequence was generated using **convenc().**

```
trellis = poly2trellis(7, [171 133]);

reference_outputs = cell(1, num_tests);

for k = 1:num_tests

    input_bits_k = test_inputs(:, k);

   reference_outputs{k} = convenc(input_bits_k, trellis);

end
```

The reference outputs were stored separately for subsequent comparison with the Simulink results.

5.3 Automated Simulink Verification

Each input sequence was applied to the Simulink model. The resulting encoded bitstream was compared against its corresponding MATLAB reference output.

The test passed when the two sequences were identical.

```
>> fprintf('\nPassed: %d/%d\n', sum(pass), num_tests);

Passed: 50/50
>> fprintf('Failed: %d/%d\n', sum(~pass), num_tests);
Failed: 0/50
```

```
>> disp(results_patterns);
    TestCase                Pattern                 InputLength    OutputLength    Pass
    ________    ____________________________________    ___________    ____________    _____

       1        {'All zeros'                  }          10             20          true
       2        {'All ones'                   }          10             20          true
       3        {'Alternating bits'           }          10             20          true
       4        {'Single 1 followed by zeros'}           10             20          true
       5        {'Mixed pattern (7 bits)'     }           7             14          true
```

Result: PASS — all 50 random test cases matched the MATLAB reference outputs.

6 . Verification Using Predefined Pattern

In addition to random testing, predefined input patterns were used to examine the encoder's behaviour under different input conditions.

Test case	Input pattern	Purpose
1	All zeros	Observe encoder behaviour with zero-valued inputs
2	All ones	Observe behaviour with continuous one-valued inputs

Test case	Input pattern	Purpose
3	Alternating zeros and ones	Examine output changes during regular input transitions
4	A single one followed by zeros	Observe the influence of a single input bit on subsequent outputs
5	Mixed binary sequence of 7 bits	Verify operation with a different input length and mixed transitions

The test patterns were defined in MATLAB:

```
test_patterns = {
                    zeros(10,1)
                    ones(10,1)
                    repmat([1;0],5,1)
                    [1; zeros(9,1)]
                    [1;0;1;1;0;1;0]
                  };
```

For each pattern, the expected encoded output was generated using **convenc()**. The corresponding Simulink output was then compared against the reference sequence.

The expected output lengths were:

Input length	Expected output length
10 bits	20 bits
7 bits	14 bits

7. Observations

The MATLAB and Simulink exploration established the following:

1. A rate-1/2 convolutional encoder was configured using constraint length 7 and generator polynomials 171 and 133 (octal).
2. The Simulink model accepted binary input sequences and generated two output bits per input bit.
3. MATLAB's convenc() function was used to generate reference output sequences.
4. An initial comparison failed because the Simulink and reference models used different input sequences. Correcting the input alignment resolved the mismatch.

5. Automated testing of 50 random input sequences produced 50 matching results.
6. Additional predefined input patterns were tested to examine encoder behaviour under different input conditions.
7. The verified MATLAB implementation provides a reference against which a future RTL implementation can be compared.

8. Conclusion

The convolutional encoder was explored and tested using MATLAB and Simulink.

A Simulink model was developed to process input binary sequences, and MATLAB's `convenc()` function was used as an independent reference. After correcting an initial input-sequence mismatch, the Simulink output matched the MATLAB reference output.

Automated verification was performed using 50 random 10-bit input sequences, with all 50 test cases passing. Additional predefined patterns were also used to investigate encoder behaviour.